

SFA Crop Book — Data Model and Provenance System

Document 05 of the SFA Product Information Pack. Audience: product planners, data engineers, ML/AI researchers, NotebookLM ingestion. **Sources:** `_aos/work_packages/S003/SFA-S003-P004-WP-CB-0/L0D200_CROP_DATA_MODEL_CANON.md` (v1.2.0 / v1.3.0 amendment); `_aos/work_packages/S003/SFA-S003-P002-WP-A/L0D400_spec.md`; `organic_market_agent/crop_book/` Python modules; `sfa_delivery/migrations/`.

Abstract. The SFA Crop Book manages agronomic knowledge for approximately 70 crops and 368 varieties grown in Israel's organic small-farm market. Rather than storing a single author's best guess, every numeric and categorical fact is tracked through a multi-source enrichment system: raw source values are tagged with source class and trust weight, reconciled by a pluggable engine that applies weighted averaging and statistical outlier rejection, and stored with per-field confidence scores, provenance metadata, and a three-state field quality signal (VALIDATED / UNVALIDATED / MISSING). The result is a knowledge base that is transparent, provenance-tagged, reproducible from the git repository, and designed to grow incrementally as new data sources are added without schema churn.

1. Design Mandate

The crop data model was canonized in 2026 (WP-CB-0, team_00 directive) after a live-data audit found eight categories of accumulated technical debt: unit chaos (e.g. temperature stored as `°C`, `celsius`, `C`), duplicate-concept fields (two yield representations, elemental and oxide nutrient forms), computed values stored as facts, categoricals with no winning-value resolution, categorical value chaos (`direct_sow` vs `direct_seed`), naming inconsistencies, and identity pollution (durations leaked into variety name fields).

The mandate: "the crop data structure must be precise, smart, and intuitive — otherwise it becomes huge technical debt. Examine the long range and prepare the data to serve the big future vision."

Seven design principles govern every decision:

1. **One concept → one canonical field → one unit → one owning layer.** No synonyms, no duplicate storage.
2. **Provenance everywhere.** Every value (numeric or categorical) carries winning source class, confidence score, source count, and observed range.
3. **Compute, do not store, what is derivable.** Derived quantities are functions of canonical facts, never persisted reconciled rows.
4. **Canonical vocabularies.** Units and categorical values come from closed, versioned enums — not free text.
5. **Crop baseline + variety override.** Each crop has exactly one default variety carrying crop-level baselines; specific varieties override only the fields that genuinely differ.
6. **Extensible by namespace, not by schema churn.** New future-module fields slot into the same typed layers under reserved namespaces — no new tables per module.

7. **Repo is the canonical source; the DB is a materialization.** Importers + source files + this canon define truth; the database is reproducible from them.

2. Field-Type Taxonomy (T1–T6)

Every crop datum is classified as exactly one of six types. The type determines the storage layer, reconciliation strategy, and consumer read path.

Type	Name	Definition	Reconciliation	Storage layer	Read path
T1	Reconciled-numeric (Fact)	Multi-source number	Trust-weighted mean or latest-op + outlier gate	<code>crop_field_enrichment</code> (<code>value_best</code>)	<code>value_best</code>
T2	Categorical-single (Attribute)	One canonical enum token	<code>hard_winner</code> by trust order	<code>crop_attribute</code> (<code>value_canonical</code>)	<code>value_canonical</code>
T3	List (Attribute)	Ordered set of canonical tokens	<code>hard_winner</code> by trust (best source's full list)	<code>crop_attribute</code> (<code>value_list jsonb</code>)	<code>value_list</code>
T4	Computed (Derived)	Function of T1/T2 facts	Not applicable — never stored	None (calculator or view)	Compute on read
T5	Identity / relational	Crop/variety identity and taxonomy	Single authored value	<code>crops</code> / <code>crop_varieties</code> / <code>crop_families</code> columns	Column
T6	Provenance	Per-fact audit metadata	Produced by reconciliation	Inside T1/T2/T3 rows	Drill-down

Layer-ownership rule (binding). A concept is stored in exactly one layer. T1 belongs in `crop_field_enrichment`, T2/T3 belong in `crop_attribute`, T5 belongs in identity columns. No concept is the source of truth in two layers simultaneously.

3. The `crop_field_enrichment` Table (T1 Numeric Facts)

This table stores the result of multi-source reconciliation for every numeric agronomic field. One row per (`variety_id`, `field_name`).

```

crop_field_enrichment
  id                bigint PK (autoincrement)
  variety_id        bigint FK → crop_varieties (ON DELETE CASCADE)
  field_name        varchar(100)  — canonical field name from the field registry
  value_min         numeric(14,6) — minimum observed (including outliers; for audit range)
  value_max         numeric(14,6) — maximum observed (including outliers)
  value_best        numeric(14,6) — reconciled best estimate (outliers excluded from blend)
  confidence_score  numeric(5,4)  — [0.0, 1.0] computed by reconciler
  source_count      int           — number of non-outlier rows that contributed
  winning_source_class varchar(20) — EX | NI | PR | OP | MK | WB | UC

```

```
computed_at      timestampz
UNIQUE (variety_id, field_name)
```

Key design choices: - `value_min` / `value_max` include statistical outliers to preserve the observed range for human audit. - `value_best` excludes outliers from the weighted blend. - `winning_source_class` records which source class determined the final value (e.g., `EX` = expert override always wins regardless of blend).

4. The `crop_attribute` Table (T2/T3 Categoricals)

Introduced alongside `crop_field_enrichment` so that categorical and list fields receive the same one-best-value + provenance treatment as numerics. One row per `(variety_id, attribute_name)`.

```
crop_attribute
  id                bigint PK
  variety_id        bigint FK → crop_varieties
  attribute_name     varchar    - canonical T2/T3 field name
  value_canonical    varchar    - T2: one canonical enum token
  value_list         jsonb      - T3: ordered array of canonical tokens
  winning_source_class varchar  - EX | NI | PR | WR | OP | MK | WB | UC
  confidence_score   numeric(5,4)
  source_count       int
  candidates         jsonb      - {source_label: value} audit trail (provenance/drill-down)
  computed_at        timestampz
  UNIQUE (variety_id, attribute_name)
```

Resolution uses `hard_winner` by the field's trust order. Crucially, raw source values are mapped through the canonical-enum map **before** a winner is chosen, so `direct_sow` → `direct_seed` and `semi_hardy` → `half_hardy` happen at import time, not during reconciliation.

5. Source Classes and Trust Tiers

The system recognizes seven source classes, registered declaratively in `organic_market_agent/crop_book/source_registry.py` (the `SOURCE_REGISTRY` dict of `SourceSpec` dataclasses).

Class	Code	Description	Default trust weight	Override behavior
Expert (team_00 overrides)	EX	Direct team_00 inputs	1.0	Hard override — skips blend entirely; always wins
Nimrod-Input	NI	Files/links provided by Nimrod (prefix <code>NI:</code>)	0.85	Hard override — wins over all blend classes
Prescriptive / MasterClass	PR	JMF Excel benchmarks, MasterClass sheets	0.70	Enters weighted blend
Operational / Tend	OP	Nimrod's historical farm records (Tend 2018–2022)	0.55	Enters weighted blend; multi-year OP is averaged first
Market index	MK	OMA price observations	0.40	Enters weighted blend

Class	Code	Description	Default trust weight	Override behavior
Web / third-party	WB	Web scraping, secondary sources	0.30	Enters weighted blend
User-Community	UC	Community-submitted (requires moderation)	0.15	Excluded from blend until moderated

Source labels for OP sources are year-qualified (`Tend_2018` through `Tend_2022`). The registry uses prefix detection for dynamic sources: any label starting with `NI:` is class NI, `OMA:` is class MK, `WB:` is class WB, `UC:` is class UC. Unknown sources fall back to WB at weight 0.20.

6. Per-Field Trust Policy (`field_policy.py`)

Each reconciled field has an explicit `FieldPolicy` controlling: - **trust_order** — the precedence list of source classes for hard-winner resolution. - **blend_strategy** — one of `weighted_mean`, `hard_winner`, or `latest_op`. - **outlier** — an `OutlierConfig` with an optional domain-function and a modified Z-score threshold (default 3.5). - **multi_year_op_mean** — when True, all OP values are averaged before entering the blend (used for yield, to prevent a single exceptional harvest year from dominating).

Example policies: - `days_to_maturity`: trust order EX→NI→PR→OP, blend `weighted_mean`, Z-threshold 3.5, domain outlier check for leaf crops. - `yield_per_bed_m`: trust order EX→NI→OP→PR→WB, blend `weighted_mean`, `multi_year_op_mean`=True, Z-threshold 3.0. - `price_documented`: trust order EX→NI→OP→MK→WB, blend `latest_op` (most recent Tend year wins for OP). - `spacing_in_row_cm`: trust order EX→NI→PR→OP→WB, blend `hard_winner`.

7. The Reconciler Algorithm

The reconciler in `organic_market_agent/crop_book/importer/reconciler.py` processes one field at a time via `reconcile_field(field_name, source_rows, name_he)`.

7.1 Step-by-step algorithm

1. **Look up the field policy** (`FIELD_POLICY.get(field_name)`). If absent, default to `hard_winner` / Z-threshold 3.5.
2. **Tag each source row** with `trust_tier`, `confidence_weight`, and `is_hard_override` from `SOURCE_REGISTRY`.
3. **Domain outlier check** — apply the field's `domain_fn` (e.g., reject DTM < 20 for leaf crops). Mark flagged rows `is_outlier_rejected=True`.
4. **Statistical outlier gate** — on the remaining rows: - If fewer than 2 rows, skip (cannot compute statistics). - Compute median and MAD (median absolute deviation). - If MAD = 0 and all values are identical: no outliers. - If MAD = 0 but values differ: use IQR fallback ($[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$). - If MAD > 0: compute modified Z-score ($0.6745 \times (x - \text{median}) / \text{MAD}$). Mark rows where $|Z| > \text{threshold}$ as `STAT_OUTLIER_REJECTED`.
5. **Compute range** — `value_min` and `value_max` include all candidates (even outlier-rejected ones) for the audit range.

6. **Hard-override check** — if any EX row is present, `value_best = EX value`, skip blend. Else if any NI row, `value_best = NI value`, skip blend.
7. **Blend** — on the remaining non-outlier `blend_rows`, apply the `blend_strategy` : -
`weighted_mean` : $\Sigma(\text{weight}_i \times \text{val}_i) / \Sigma(\text{weight}_i)$. - `hard_winner` : value from the highest-trust-order class present. - `latest_op` : OP value with the lexicographically latest source label (`Tend_2022 > Tend_2021 > ...`).
8. **Compute confidence score** — formula accounts for source diversity (number of distinct classes present vs possible) and low spread: - 0 blend rows → `confidence_score = 0.0` . - 1 blend row → `confidence_score = 0.15` . - Otherwise: `coverage = classes_present / classes_possible`; `spread = std_dev / mean`; `confidence_score = coverage × (1.0 - min(spread, 1.0) × 0.5)`, clamped to `[0.0, 1.0]` .

7.2 Enrichment runner

`organic_market_agent/crop_book/importer/enrichment_runner.py` iterates over all varieties (optionally filtered), loads their source values for the whitelisted fields, calls `reconcile_field`, and upserts into `crop_field_enrichment`. The upsert key is `(variety_id, field_name)` — idempotent.

8. Field-State Model (VALIDATED / UNVALIDATED / MISSING)

Every field in the agronomy whitelist has a `field_state` computed at publish time. The threshold constant is $\tau = 0.40$.

State	Condition
VALIDATED	<code>winning_source_class</code> is EX or NI, OR <code>confidence_score</code> ≥ 0.40
UNVALIDATED	A <code>crop_field_enrichment</code> row exists but neither condition above is met
MISSING	No <code>crop_field_enrichment</code> row exists for this field on this variety

For categorical attributes (T2/T3): **VALIDATED** if a non-null `value_canonical` or non-empty `value_list` is present; **MISSING** otherwise.

The `field_state` map is embedded in each variety's `payload_json` on the delivery tier, enabling the UI to show provenance cues: a **VALIDATED** fact can show a confidence indicator, an **UNVALIDATED** one can prompt "we're not certain — help us verify", and a **MISSING** field can trigger a "fill this gap" call to action.

9. Canonical Vocabularies

9.1 Unit Registry (one string per dimension)

All unit strings are normalized to canonical forms. The migration phase resolves every variant observed in the live database to leave zero non-canonical unit strings.

Dimension	Canonical unit	Normalized from
Temperature	°C	celsius , C
Duration (days)	days	—
Duration (weeks)	weeks	—
Length	cm	—
Yield (per bed-meter)	kg_per_bed_m	kg/m , bare kg (sloppy unit string on per-bed-m values)
Nutrient load	kg_per_ha	kg/ha
Price	ILS_per_<unit>	ILS/kg , ILS/bunch , ILS/unit
Ratio/percent	pct	%
Acidity	pH	blank/NULL
Seed density	seeds_per_g	seeds/g
Count	count	rows , blank/NULL
Yield (per m²) — derived only	kg_per_m2	Not stored; converted from kg_per_bed_m at runtime via bed_width AssumptionField

The unit for each canonical field name is recorded in `organic_market_agent/crop_book/canon/units.py` and `field_registry.py`. Units are keyed to field names — they are not re-spelled per row.

9.2 Canonical Enums (T2/T3 Closed Sets)

Attribute	Canonical tokens	Collapsed from
planting_method	direct_seed , transplant , seed_tuber , slip , cutting	direct_sow → direct_seed
frost_tolerance_class	hardy , half_hardy , tender , very_tender	semi_hardy → half_hardy ; half-hardy → half_hardy
growth_cycle	annual , biennial , perennial	—
category	vegetables , herbs , fruits , fruit_trees	—
harvest_unit	kg , bunch , head , case , unit , seedling	—
harvest_stage	full_size , baby_leaf , head , plant_sale , seed	—
storage_ethylene_sensitivity	none , low , medium , high	normalize free text → these 4
irrigation_type	drip , sprinkler , mixed	—
root_depth_class	shallow , medium , deep	—
needs_summer_shade	none , shade_30 , shade_40 , shade_50	Israel-specific (NI-sourced)

Attribute	Canonical tokens	Collapsed from
sowing_months / transplant_months	Array of ints 1-12	CSV "2,3,5" → [2, 3, 5]

CLOSED-ENUM attributes reject any out-of-set token at import (logged and routed to data quality). Open-vocabulary attributes (variety_provider , rootstock_variety , common_pests , foliar_feeding_program , unit_size) accept free text but apply trim/case/dedup normalization and still carry full provenance in crop_attribute.candidates .

9.3 Naming Convention

Field names follow the pattern <concept>[_<qualifier>][_<unit-suffix-only-when-it-disambiguates>], in snake_case, without avg_ or default_ prefixes (averaging is a reconciliation detail, not part of the name). Examples: yield_per_bed_m , spacing_in_row_cm , days_to_maturity , days_in_nursery .

10. Canonical Field Registry

The complete field registry is implemented in organic_market_agent/crop_book/canon/field_registry.py as a FIELD_REGISTRY dict mapping current (or canonical) names to FieldMeta dataclasses with canonical , field_type , layer , disposition , and unit attributes.

10.1 Key T1 Reconciled-Numeric Facts

Canonical field	Old name	Unit	Notes
days_to_maturity	—	days	Core planning field
spacing_in_row_cm	in_row_spacing_cm	cm	Renamed
rows_per_bed	—	count	JM Fortier farm native
yield_per_bed_m	avg_yield_per_bed_m	kg_per_bed_m	Canonical yield; bare kg unit string normalized at migration
price_documented	documented_price	ILS_per_<unit>	Renamed
seeds_per_g	seeds_per_gram	seeds_per_g	Renamed
nutrient_removal_{n,p,k,ca,mg}_kg_per_ha	*_kg_ha	kg_per_ha	Elemental only (oxide forms are T4 derived)
germination_temp_{min,opt,max}_c	—	°C	Unit-normalized
storage_temp_c_{min,max} , storage_rh_pct_{min,max}	—	°C , pct	—
storage_life_days	—	days	

Canonical field	Old name	Unit	Notes
			Text form is T4 derived/dropped
days_in_nursery	days_in_gh_total	days	Sow → field-transplant total; renamed
succession_interval_weeks	—	weeks	—
drip_lines_per_bed	—	count	MIG2 addition
labor_rate_harvest , labor_rate_wash	—	units_per_hr	MIG2 addition
plantings_per_season , harvest_weeks_span	—	count , weeks	MIG2 addition

10.2 T4 Computed (never stored)

Derived field	Formula
yield_per_m2	yield_per_bed_m / bed_width (AssumptionField default 0.8 m)
plants_per_m2	function of rows_per_bed , spacing_in_row_cm , bed_width
Oxide nutrients p2o5 , k2o	$p \times 2.29$; $k \times 1.205$
avg_revenue_per_bed_m	yield_per_bed_m × price_documented

11. The 13-Topic Taxonomy (CROP_TOPICS)

The canonical topic ordering applies to both the schema (field grouping) and the UI (section order at all display depths). It follows the structure of the JMF MasterClass sheets — the ordering growers already know.

#	Key	Hebrew	Topic
1	varieties	זנים	Varieties
2	spacing	מרווח ופריסה	Spacing & layout
3	equipment	ציוד וכיוון	Equipment & tuning
4	soil	קרקע ודישון	Soil & fertilization
5	bedprep	הכנת ערוגה	Bed preparation
6	sowing	זריעה/שתילה	Sowing / transplanting
7	irrigation	השקיה	Irrigation
8	care	טיפול ועיבוד	Care & weeding
9	pest	מזיקים ומחלות	Pests & diseases
10	harvest	קציר	Harvest

#	Key	Hebrew	Topic
11	storage	שטיפה ואחסון	Washing & storage
12	succession	רצף וחברה	Succession & companions
13	yield_inc	יבול/הכנסה	Yield / revenue (calculator-facing)

The taxonomy is implemented as the `CROP_TOPICS` constant in `organic_market_agent/crop_book/canon/topics.py`, consumed by both the schema field registry and the delivery-tier UI so ordering has one single source of truth.

12. Data Sources

12.1 JMF MasterClass (PR class — weight 0.70)

The primary prescriptive source. Two data streams: - **JMF Excel benchmarks** — per-crop price and yield reference data imported by `organic_market_agent/crop_book/importer/jmf.py`. - **MasterClass sheets** — 37 markdown-processed crop sheets in `documentation/jmf_masterclass_crop_sheets/`, covering agronomic detail for each crop and variety. Imported via `load_masterclass_sheets.py` to populate irrigation type, root depth class, harvest windows, partial pest/foliar data, and more.

JMF data is Israeli market-garden scale, following the JM Fortier market gardening methodology adapted for Israel. It is the backbone of the crop book's agronomic baselines.

12.2 Tend Israel Farm Records (OP class — weight 0.55)

Nimrod's operational farm data from the Tend platform, covering 2018–2022. Tables currently ingested: `CROP_PLAN` (529 rows), `PRODUCT_SOLD`, `HARVESTS` (939 rows — 2022 subset). Multi-year OP values for yield fields are averaged before entering the blend, reducing the weight of any single exceptional year.

Not yet ingested: `CROPAVAILABILITY`, `GREENHOUSE_PLAN`, `LOCATIONS`, `ORDERS_RAW_DATA`, and cross-year data for 2018–2021. These represent high-value future enrichment opportunities.

12.3 Nimrod-Input Validation (NI class — weight 0.85, hard override)

Expert validation by Nimrod via a purpose-built static HTML console (`scripts/build_crop_gap_console.py`). The console generates one record per `(crop × missing field)` gap, grouped by the 13-topic taxonomy, pre-filled with best-effort defaults (PR parse where available, else agronomic defaults). Nimrod confirms/edits/skips each entry and exports a JSON. The JSON is ingested as NI class by `scripts/ingest_nimrod_validation.py`. NI class acts as a hard override — it wins over PR and OP in all fields.

12.4 Expert Overrides (EX class, weight 1.0 hard override)

Direct `team_00` inputs, stored with source label `team_00`. These are the highest-trust entries and skip the weighted blend entirely. Currently applied to individual crop corrections (e.g. arugula DTM override).

12.5 Web Sources and Market Index (WB / MK classes)

Web-scraped agronomic data (WB, weight 0.30) and OMA market price observations (MK, weight 0.40) are registered in the source registry. WB and MK importers are design-registered (sentinel entries in `SOURCE_REGISTRY`) but not yet wired to active importers in the crop-book enrichment path.

13. Crop Identity, Granularity, and the Default-Variety Pattern

The live database contains 70 crops, 368 varieties, and 2,061 source values. Each crop has at least one variety; 70 of 70 crops have exactly one default variety (`is_default=True`) that carries the crop-level baseline for every field.

Inheritance rule: variety value (from `crop_field_enrichment` / `crop_attribute` keyed to that variety's `id`) takes precedence; if absent, the default-variety value is used. This is enforced at the consumer read layer — the database itself does not duplicate values.

Identity columns (`crop_varieties`): `name_he`, `name_en`, `is_default`, `is_grafted`, `harvest_unit_default`, `icon_url`, and seeder settings (`seeder`, `seeder_front_gear`, `seeder_rear_gear`, `seeder_roller_plate`, `seeder_settings`). These are T5 identity fields, single-authored, and are never enriched or reconciled. The seeder column is the SSoT for the seeder model name (e.g., "JANG 3X"); the separate gear and roller plate columns are operational tuning values.

Data quality note: a data-quality pass purges duration text that leaked into variety `name_he` fields (values like "45 חודשים 3", "יום") — these are durations, not variety names, and migrated to appropriate numeric fields.

14. AssumptionFields (User-Adjustable Planning Inputs)

Crop-book calculators distinguish between **book values** (from `crop_field_enrichment.value_best`) and **assumption values** — user-adjustable planning inputs that affect derived quantities. Assumptions are registered in `organic_market_agent/crop_book/assumptions.py` as frozen `Assumption` dataclasses and are embedded in each variety's `payload_json` on the delivery tier so the JS UI and Python calculator layer share one source of truth.

Key	Default	Unit	Purpose
<code>germination_rate</code>	0.90	fraction	Fraction of seeds expected to germinate
<code>bed_width</code>	0.80	m	Bed width (JM Fortier standard, 80 cm) — also used to derive <code>yield_per_m2</code>
<code>oversow</code>	1.10	x	Seed quantity safety factor (10% extra)
<code>std_bed_length_m</code>	30.0	m	Standard bed length for bed-count calculations
<code>compost_N_pct</code>	0.015	fraction	Compost nitrogen content
<code>hardiness_offset</code>	table	—	Temperature offset table for hardiness zone adjustments
<code>tray_cells</code>	table	—	Standard tray cell counts by tray size

Key	Default	Unit	Purpose
price_override	None	ILS	User-supplied price override for revenue calculations

Assumptions are NOT per-crop book data and do not affect a crop's COMPLETE/PARTIAL provenance status.

15. 14 Calculators (Crop Book v1)

Pure-function calculators implemented in `organic_market_agent/crop_book/calculators.py`. Each raises `CalcUnavailable(missing_field)` when a required book value is `None` (missing from `crop_field_enrichment`). Result dataclasses are frozen.

#	Calculator	Required book fields	Output
1	Seed quantity	seeds_per_g , plant population inputs	plants , seeds , grams
2	Nursery tray count	plant count, tray cell assumption	trays , tray_sow_date
3	Sowing / transplant date	days_in_nursery	sow_date , field_set_date
4	Harvest window	days_to_maturity , harvest_window_max_days	harvest_start , harvest_end
5	Beds for target yield	yield_per_bed_m	bed_meters , beds
6	Revenue estimate	yield_per_bed_m , price_documented	yield_kg , revenue
7	Plant population	rows_per_bed , spacing_in_row_cm , bed_width	plants_per_m2 , grid
8	Fertilizer requirement	nutrient removal fields, compost assumption	NPK quantities
9	Germination temperature suitability	germination_temp_*_c	suitability rating
10	Storage life estimate	storage_life_days , storage_temp_c_*	storage guidance
11	Succession planting schedule	succession_interval_weeks , harvest_weeks_span	succession plan
12	Seed viability by year	crop-specific viability data	viability rating
13	Labor estimate	labor_rate_harvest , labor_rate_wash	hours estimate
14	Cover crop period	cover crop reference data	cover crop windows

Calculators are pure Python functions — no database, no I/O, no globals. The JavaScript UI mirror (`cropbook-v1.js`, the `CALC` object) must stay in parity with `calculators.py`.

16. Delivery-Side Data Representation

When data is published to the delivery tier (MySQL on `sfa.nimrod.bio`), it is transformed from the Postgres normalized form to a hybrid schema:

- **Top-level columns** on the `crops` table: `id`, `slug`, `hebrew_name`, `scientific_name`, `family_id`, `family_name_he`, `category`, `season`, `dtm_min`, `dtm_max`, `last_pushed_at`. These support grid filters and sorts.
- **payload_json** carries everything else: identity block, planting calendar, agronomy facts (T1 numeric + T2/T3 categoricals merged), harvest stats, postharvest storage, companion matrix, knowledge notes, variety summary, and the ASSUMPTIONS registry.
- **crop_field_enrichment mirror table** (delivery tier, migration `004_crop_field_enrichment.sql`): one row per (`crop_id`, `field_name`) using the representative variety, with `value_best`, `unit`, `field_state`, `winning_source_class`, and `confidence_score`. This table is consumed directly by the Assumptions/Calculators controller (`AssumptionsController.php`) and by the calculator hub (`HubController::calc()`).
- **crop_attribute mirror table** (delivery tier, migration `005_crop_attribute.sql`): one row per (`crop_id`, `attribute_key`) with `value_canonical`, `value_list`, and `field_state`.

The delivery-side `field_state` (VALIDATED / UNVALIDATED / MISSING at $\tau=0.40$ or EX/NI class) is computed at push time in `sfa_ingest_push.py` and stored in both the `payload_json` field-state map and the `crop_field_enrichment.field_state` column.

17. Extensibility and the Future-Vision Namespace

Future modules slot new fields into the existing typed layers under reserved name prefixes — no new tables per module:

Module	Namespace	Storage layer
Planner (CB-2)	<code>plan_</code>	T1 enrichment / T4 computed
Tasks (CB-3)	<code>task_</code>	<code>crop_task_templates</code> table (already exists)
Sales/POS (CB-4)	<code>sale_</code>	T1 enrichment / T4 computed
Tend integration (CB-5)	<code>op_</code>	OP-class source values → enrichment

The structural guarantee: a new module registers fields of types T1–T5 with canonical names, units, and enums, and the existing enrichment/attribute/computed machinery serves them without schema changes.

Document 05 — authored 2026-06-03 by team_100 for the SFA Product Information Pack. Sources: LOD200_CROP_DATA_MODEL_CANON.md v1.2.0/v1.3.0; LOD400_spec.md (WP-A); sfa_ingest_push.py; enrichment_models.py; source_registry.py; field_policy.py; assumptions.py; calculators.py; sfa_delivery/migrations/.